



# CSE 1112: Structured Programming Language Laboratory

**Sidratul Tanzila Tasmi**

Lecturer, Department of Computer Science and Engineering  
United International University

# LOOP



Syntax:

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

Expression 1 is executed (one time) before the execution of the code block.

Expression 2 defines the condition for executing the code block.

Expression 3 is executed (every time) after the code block has been executed.

# Print Numbers from 1 to 4



The example below will print the numbers 0 to 4:

Example

```
int i;
```

```
for (i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

# For Loop



Initialization Expression: In this expression, we assign a loop variable or loop counter to some value. for example: `int i=1;`

Test Expression: In this expression, test conditions are performed. If the condition evaluates to true then the loop body will be executed and then an update of the loop variable is done. If the test expression becomes false then the control will exit from the loop. for example, `i<=9;`

Update Expression: After execution of the loop body loop variable is updated by some value it could be incremented, decremented, multiplied, or divided by any value.

# For Loop Code to Print Hello World 10 times



```
#include <stdio.h>

int main()
{
    int i = 0;

    for (i = 1; i <= 10; i++)
    {
        printf( "Hello World\n");
    }
    return 0;
}
```

## Output

[illegible]

# Reverse Counting in for Loop



```
for(int i = 10; i >= 1; i--) {  
    printf("%d\n", i);  
}
```

# Multiple variables in a for loop



```
for(int i = 0, j = 10; i < j; i++, j--) {  
    printf("i = %d, j = %d\n", i, j);  
}
```

# Infinite for Loop



This code will run forever

```
for(;;) {  
    printf("This runs forever\n");  
}
```

# Omitting Initialization

```
int i = 0;  
for(; i < 5; i++) {  
    printf("%d\n", i);  
}
```



# Loop With Floating-Point Counter

```
for(float x = 0.0; x <= 1.0; x += 0.2) {  
    printf("%.1f\n", x);  
}
```





# Break statement

The break in C is a loop control statement that breaks out of the loop when encountered.

It can be used inside loops or switch statements to bring the control out of the block.

The break statement can only break out of a single loop at a time.

Syntax of break in C:

```
break;
```



# Continue Statement

The C continue statement resets program control to the beginning of the loop when encountered.

As a result, the current iteration of the loop gets skipped and the control moves on to the next iteration.

Statements after the continue statement in the loop are not executed.

Syntax of continue in C: The syntax of continue is just the continue keyword placed wherever we want in the loop body.

```
continue;
```

# Break and Continue in for Loop



– continue

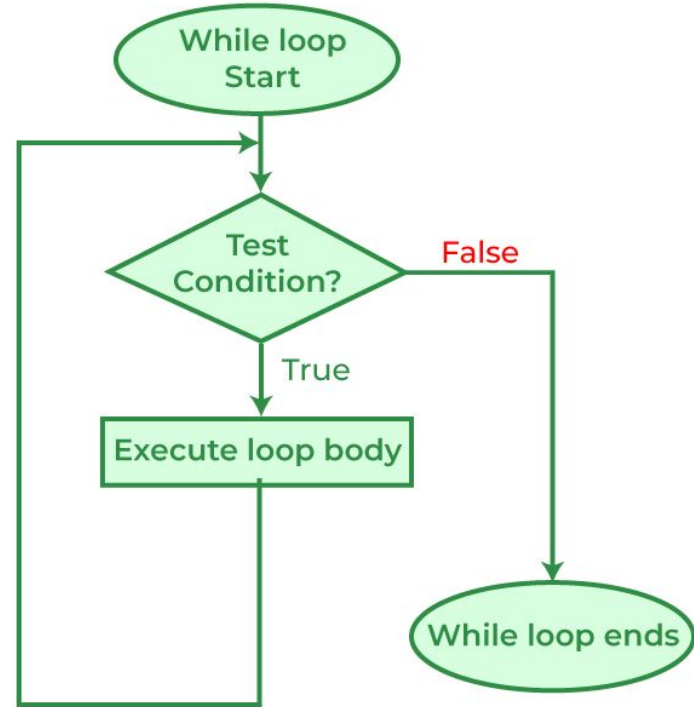
```
for(int i = 1; i <= 5; i++) {  
    if(i == 3) continue;  
    printf("%d\n", i);  
}
```

-break

```
for(int i = 1; i <= 5; i++) {  
    if(i == 4) break;  
    printf("%d\n", i);  
}
```

# While Loop

In the While loop, the execution is terminated on the basis of the test condition. If the test condition becomes false then it breaks from the while loop else body is executed.



# Syntax for while loop



Syntax:

```
initialization_expression;

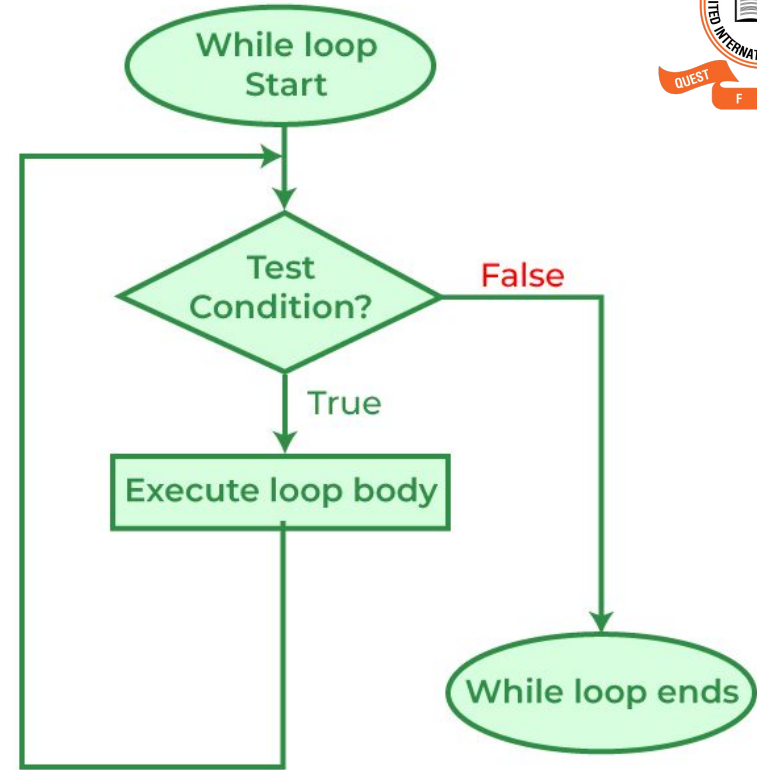
while (test_expression)
{
    // body of the while loop

    update_expression;
}
```

```
#include <stdio.h>
int main()
{
    int i=0;
    while(i<10){
        printf( "Hello World\n");
        i++;
    }
    return 0;
}
```

# while loop

- In the While loop, the execution is test condition. If the test condition is true, the body of the while loop is executed. If the test condition is false, the execution of the while loop ends.



# while loop



```
#include <stdio.h>
int main()
{
    int i=0;
    while(i<10){
        printf( "Hello World\n");
        i++;
    }
    return 0;
}
```

# while loop

```
#include <stdio.h>
int main()
{
    // Initialization expression
    int i = 2;

    // Test expression
    while(i < 10)
    {
        // loop body
        printf( "Hello World\n");

        // update expression
        i++;
    }

    return 0;
}
```

## Output

```
/tmp/PJHEGj0peP.o
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
|
```

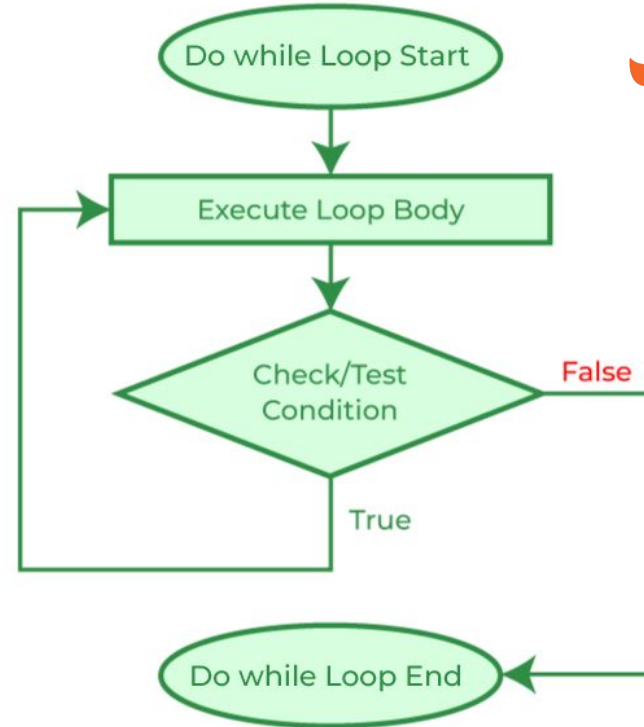
# Exit controlled loop

In Exit controlled loops **the test condition is evaluated at the end of the loop body**. The loop body will execute **at least once**, irrespective of whether the condition is true or false. **do-while** Loop is Exit Controlled loop.

|                  |                                                                        |
|------------------|------------------------------------------------------------------------|
| do-while<br>loop | do-while first executes the body and then the condition check is done. |
|------------------|------------------------------------------------------------------------|

# do-while

- The do-while loop is similar to a while loop. The difference lies in the do-while loop test condition. In the do-while loop, the body is executed at least once irrespective of the test condition.



# do-while

Syntax:

```
initialization_expression;
do
{
    // body of do-while loop

    update_expression;

} while (test_expression);
```

```
#include <stdio.h>
int main()
{
    int i=0;
    do{
        printf( "Hello World\n");
        i++;
    }while(i<10);
    return 0;
}
```

# do-while

```
#include <stdio.h>
int main()
{
    // Initialization expression
    int i = 2;

    do
    {
        // loop body
        printf( "Hello World\n");

        // Update expression
        i++;

        // Test expression
    } while (i < 10);

    return 0;
}
```

## Output

```
/tmp/PJHEGjOpeP.o
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```



# do-while



```
#include <stdio.h>
int main()
{
    // Initialization expression
    int i = 2;

    do
    {
        // loop body
        printf( "Hello World\n");

        // Update expression
        i++;

        // Test expression
    } while (i < 1);

    return 0;
}
```

## Output

```
/tmp/PJHEGj0peP.o
Hello World
```

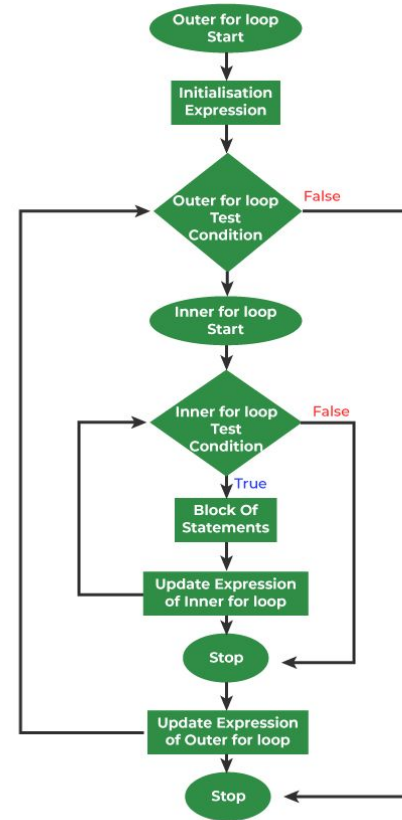
# Nested Loop [Ref:

<https://www.geeksforgeeks.org/nested-loops-in-c-with-examples/>]



- A nested loop means a loop statement inside another loop statement.
- That is why nested loops are also called “loop inside loops”.
- We can define any number of loops inside another loop.

# Nested for Loop



# Nested for loop

Syntax:

```
for ( initialization; condition; increment ) {  
  
    for ( initialization; condition; increment ) {  
  
        // statement of inside loop  
    }  
  
    // statement of outer loop  
}
```

# Example

```
#include <stdio.h>
int main() {
    int i, j;
    for (i = 1; i <= 5; i++) {
        printf("i=%d ", i);
        for (j = 1; j <= i; j++) {
            printf("j=%d ", j);
        }
        printf("\n");
    }
    return 0;
}
```

```
i=1 j=1
i=2 j=1 j=2
i=3 j=1 j=2 j=3
i=4 j=1 j=2 j=3 j=4
i=5 j=1 j=2 j=3 j=4 j=5
```

# Nested While Loop

A nested while loop refers to any type of loop that is defined inside a 'while' loop.

Syntax:

```
while(condition) {  
  
    while(condition) {  
  
        // statement of inside loop  
    }  
  
    // statement of outer loop  
}
```

# Example

```
i=1 j=1
i=2 j=1 j=2
i=3 j=1 j=2 j=3
i=4 j=1 j=2 j=3 j=4
i=5 j=1 j=2 j=3 j=4 j=5
```

```
#include <stdio.h>
int main() {
    int i, j;
    i=1;
    while (i <= 5) {
        printf("i=%d ", i);
        j=1;
        while (j <= i) {
            printf("j=%d ", j);
            j++;
        }
        printf("\n");
        i++;
    }
    return 0;
}
```

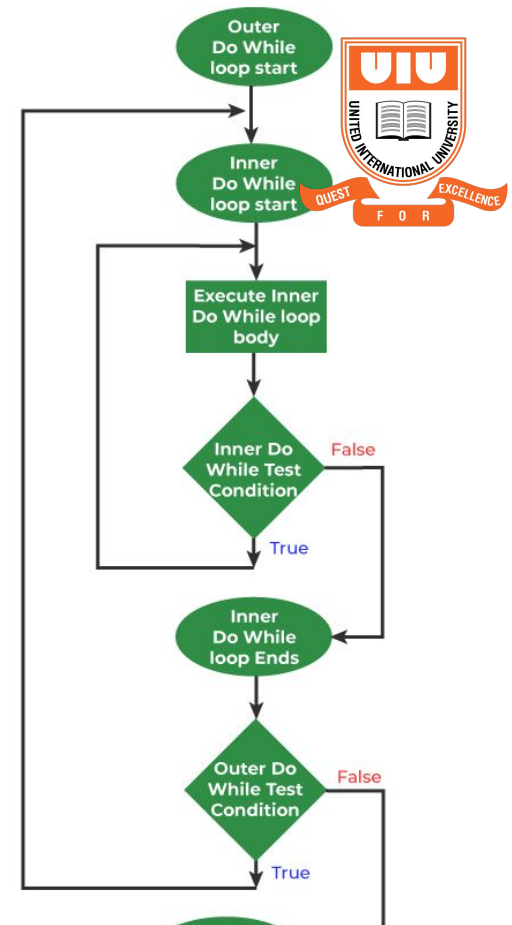


# Nested do while loop

A nested do-while loop refers to any type of loop that is nested inside a do-while loop.

Syntax:

```
do{  
  
    do{  
  
        // statement of inside loop  
    }while(condition);  
  
    // statement of outer loop  
}while(condition);
```



# Example

```
i=1 j=1
i=2 j=1 j=2
i=3 j=1 j=2 j=3
i=4 j=1 j=2 j=3 j=4
i=5 j=1 j=2 j=3 j=4 j=5
```

```
#include <stdio.h>
int main() {
    int i, j;
    i=1;
    do {
        printf("i=%d ", i);
        j=1;
        do {
            printf("j=%d ", j);
            j++;
        } while (j <= i);
        printf("\n");
        i++;
    } while (i <= 5);
    return 0;
}
```



# for-while-do while



Syntax:

```
do{  
  
    while(condition) {  
  
        for ( initialization; condition; increment ) {  
  
            // statement of inside for loop  
        }  
  
        // statement of inside while loop  
    }  
  
    // statement of outer do-while loop  
}while(condition);
```

# Example



```
do while i=1 while j=1
do while i=2 while j=1 while j=2
do while i=3 while j=1 while j=2 while j=3
do while i=4 while j=1 while j=2 while j=3 while j=4
do while i=5 while j=1 while j=2 while j=3 while j=4 while j=5
```

```
#include <stdio.h>
int main() {
    int i, j;
    i=1;
    do {
        printf("do while i=%d ", i);
        j=1;
        while (j <= i) {
            printf("while j=%d ", j);
            j++;
        }
        printf("\n");
        i++;
    } while (i <= 5);
    return 0;
}
```



Thank you!